# Entity Relationship Document

*Campus Complaint & Grievance Tracker*

## Note on the ER Diagram

The ER diagram follows Chen notation (rectangles for entities, ellipses for attributes, diamonds for relationships, double borders for weak entities, double lines for total participation, and single lines for partial participation).

## 1. Entities and Attributes

Underlined items are primary keys. Attribute types follow standard ER notation: simple (plain ellipse), composite (an attribute broken into sub-attributes), multivalued (double ellipse), derived (dashed ellipse), and foreign keys establishing relationship references.

### Student

| Attribute | Type | Notes |
|---|---|---|
| Student_ID | Simple, Key (PK) | Uniquely identifies a student |
| Name | Composite | Broken into First_Name and Last_Name |
| Email | Simple | Login/contact email |
| Phone_Numbers | Multivalued | A student may have more than one contact number |
| Room_No | Simple | Hostel room number |
| Total_Complaints_Filed | Derived | Computed by counting the student's rows in Complaint — not stored directly |
| Hostel_ID | Foreign Key (FK) | References Hostel(Hostel_ID) via *Resides_In* |

### Staff

| Attribute | Type | Notes |
|---|---|---|
| Staff_ID | Simple, Key (PK) | Uniquely identifies a staff member |
| Name | Composite | Broken into First_Name and Last_Name |
| Designation | Simple | Staff role or designation |
| Email | Simple | Official contact email |
| Dept_ID | Foreign Key (FK) | References Department(Dept_ID) via *Employs* |

## Department

| Attribute | Type | Notes |
|---|---|---|
| Dept_ID | Simple, Key (PK) | Uniquely identifies a department |
| Dept_Name | Simple | Department name (e.g., Estate, IT, Hostel Office) |
| Dept_Head | Simple | Head of Department name |
| *(Foreign Keys)* | *None* | Independent master entity; referenced by Staff and Category |

## Category

| Attribute | Type | Notes |
|---|---|---|
| Category_ID | Simple, Key (PK) | Uniquely identifies a grievance category |
| Category_Name | Simple | Category name (e.g., Electrical, Plumbing, Mess, Internet) |
| SLA_Days | Simple | Days allowed before auto-escalation |
| Dept_ID | Foreign Key (FK) | References Department(Dept_ID) via *Owns* |

## Hostel

| Attribute | Type | Notes |
|---|---|---|
| Hostel_ID | Simple, Key (PK) | Uniquely identifies a hostel |
| Hostel_Name | Simple | Name of hostel |
| Warden_ID | Foreign Key (FK) | References Staff(Staff_ID) via *Wardens* (1:1 relationship) |

## Complaint

| Attribute | Type | Notes |
|---|---|---|
| Complaint_ID | Simple, Key (PK) | Uniquely identifies a complaint |
| Description | Simple | Detailed grievance description |
| Status | Simple | Open / Assigned / In-Progress / Resolved / Escalated |
| Created_At | Simple | Timestamp when complaint was lodged |
| Updated_At | Simple | Timestamp of last modification or status change |
| Student_ID | Foreign Key (FK) | References Student(Student_ID) via *Files* |
| Category_ID | Foreign Key (FK) | References Category(Category_ID) via *Classified_As* |

### StatusHistory (Weak Entity)

Modeled as a weak entity because a history row has no independent meaning without the Complaint it belongs to — it is identified only in combination with Complaint_ID.

| Attribute | Type | Notes |
|---|---|---|
| History_Num | Partial Key | Only unique within a given complaint; combined with Complaint_ID to form the full key |
| Old_Status | Simple | Status prior to change |
| New_Status | Simple | Status after change |
| Changed_At | Simple | Timestamp of status update |
| Complaint_ID | Foreign Key (FK) | References Complaint(Complaint_ID) via *Has* (identifying relationship) |

## 2. Relationships, Cardinality & Participation Constraints

Participation is 'total' (every instance on that side MUST take part — drawn as a bold/double line) or 'partial' (participation is optional — drawn as a thin single line).

| Relationship | Entities | Attributes | Card. | Participation |
|---|---|---|---|---|
| **Files** | Student ↔ Complaint | none | 1 : N | Student: partial (a student may file zero complaints). Complaint: total (every complaint must be filed by a student). |
| **Classified_As** | Complaint ↔ Category | none | N : 1 | Complaint: total (every complaint must have a category). Category: partial (a category may have zero complaints so far). |
| **Owns** | Department ↔ Category | none | 1 : N | Department: partial. Category: total (every category must belong to a department). |
| **Employs** | Department ↔ Staff | none | 1 : N | Department: partial. Staff: total (every staff member belongs to a department). |
| **Resides_In** | Student ↔ Hostel | none | N : 1 | Student: total (every student is assigned a hostel). Hostel: partial. |
| **Wardens** | Staff ↔ Hostel | none | 1 : 1 | Staff: partial (not every staff member is a warden). Hostel: total (every hostel must have exactly one current warden). |

| Relationship | Entities | Attributes | Card. | Participation |
|---|---|---|---|---|
| **Assigned_To** | Staff ↔ Complaint | Assigned_At, Unassigned_At | N : M | Staff: partial. Complaint: partial (a brand-new 'Open' complaint has no assignment yet). |
| **Has** | Complaint ↔ StatusHistory | none (identifying relationship) | 1 : N | Complaint: partial. StatusHistory: total (a history row cannot exist without its complaint). |

**Note:** *Assigned_To* carries its own attributes (*Assigned_At*, *Unassigned_At*) because the relationship itself has data that doesn't belong to either Staff or Complaint alone — this is exactly the case where an N:M relationship becomes its own table during schema conversion (Step 3).

## 3. ER Diagram → Relational Schema (with PK / FK)

Each entity becomes a table. 1:N relationships are implemented by placing the 'many' side's foreign key pointing to the 'one' side. The N:M relationship (*Assigned_To*) becomes its own table. The weak entity (*StatusHistory*) takes a composite primary key combining its partial key with its owner's key.

| Table Name | Primary Key (PK) | Foreign Key (FK) & References | Other Attributes |
|---|---|---|---|
| **Student** | Student_ID | Hostel_ID → Hostel.Hostel_ID | First_Name, Last_Name, Email, Room_No |
| **Student_Phone** | (Student_ID, Phone_Number) | Student_ID → Student.Student_ID | *(None — composite key table)* |
| **Staff** | Staff_ID | Dept_ID → Department.Dept_ID | First_Name, Last_Name, Designation, Email |
| **Department** | Dept_ID | *(None)* | Dept_Name, Dept_Head |
| **Category** | Category_ID | Dept_ID → Department.Dept_ID | Category_Name, SLA_Days |
| **Hostel** | Hostel_ID | Warden_ID → Staff.Staff_ID | Hostel_Name |
| **Complaint** | Complaint_ID | Student_ID → Student.Student_ID<br>Category_ID → Category.Category_ID | Description, Status, Created_At, Updated_At |
| **StatusHistory** | (Complaint_ID, History_Num) | Complaint_ID → Complaint.Complaint_ID | Old_Status, New_Status, Changed_At |
| **Staff_Assignment** | (Staff_ID, Complaint_ID, Assigned_At) | Staff_ID → Staff.Staff_ID<br>Complaint_ID → Complaint.Complaint_ID | Unassigned_At |

**Standard Relational Notation:**

```
Student(Student_ID PK, First_Name, Last_Name, Email, Room_No, Hostel_ID FK ->
Hostel.Hostel_ID)
  -- Phone_Numbers (multivalued) is pulled out into its own table:
Student_Phone(Student_ID FK -> Student.Student_ID, Phone_Number, PRIMARY KEY(Student_ID,
Phone_Number))

Staff(Staff_ID PK, First_Name, Last_Name, Designation, Email, Dept_ID FK ->
Department.Dept_ID)

Department(Dept_ID PK, Dept_Name, Dept_Head)

Category(Category_ID PK, Category_Name, SLA_Days, Dept_ID FK -> Department.Dept_ID)

Hostel(Hostel_ID PK, Hostel_Name, Warden_ID FK -> Staff.Staff_ID)

Complaint(Complaint_ID PK, Description, Status, Created_At, Updated_At,
          Student_ID FK -> Student.Student_ID, Category_ID FK -> Category.Category_ID)

StatusHistory(Complaint_ID FK -> Complaint.Complaint_ID, History_Num,
              Old_Status, New_Status, Changed_At,
              PRIMARY KEY(Complaint_ID, History_Num))   -- weak entity: composite key

Staff_Assignment(Staff_ID FK -> Staff.Staff_ID, Complaint_ID FK -> Complaint.Complaint_ID,
                 Assigned_At, Unassigned_At,
                 PRIMARY KEY(Staff_ID, Complaint_ID, Assigned_At))   -- N:M relationship
table
```

- **Why Total_Complaints_Filed doesn't appear in any table:** it's a derived attribute — it is calculated on demand with `COUNT(*)` on Complaint rather than stored, which avoids having to keep a stored number in sync every time a complaint is added.
- **Why Warden_ID lives on Hostel, not a separate table:** Wardens is a 1:1 relationship, so the foreign key can sit on either side; placing it on Hostel keeps 'which hostel has which warden' a single lookup.

## 4. Functional Dependencies

Listed per table — these are what justify each table already being in at least 3NF.

```
Student:            Student_ID → First_Name, Last_Name, Email, Room_No, Hostel_ID
Student_Phone:      Student_ID, Phone_Number → (no other non-key attribute; key is the
full combination)
Staff:              Staff_ID → First_Name, Last_Name, Designation, Email, Dept_ID
Department:         Dept_ID → Dept_Name, Dept_Head
Category:           Category_ID → Category_Name, SLA_Days, Dept_ID
Hostel:             Hostel_ID → Hostel_Name, Warden_ID
Complaint:          Complaint_ID → Description, Status, Created_At, Updated_At,
Student_ID, Category_ID
StatusHistory:      Complaint_ID, History_Num → Old_Status, New_Status, Changed_At
Staff_Assignment:   Staff_ID, Complaint_ID, Assigned_At → Unassigned_At

Transitive dependency deliberately avoided:
  Complaint_ID → Student_ID → Hostel_ID   (so Hostel_ID is NOT stored again on Complaint)
  Category_ID → Dept_ID                   (so Dept_ID is NOT stored again on Complaint)
```

Because every non-key attribute in every table depends only on that table's whole primary key — and nothing transitively through another non-key attribute — the schema satisfies **3NF**.